PROGRAMMING & CS FUNDAMENTALS

Design Patterns I: Creational Visual Guide

Who makes the objects —
five ways to control creation.

ANIMATED WALKTHROUGH
Step through pages to play the animation

STEP 1/6

Singleton: One Instance, Global Access

A class that refuses to be built twice.

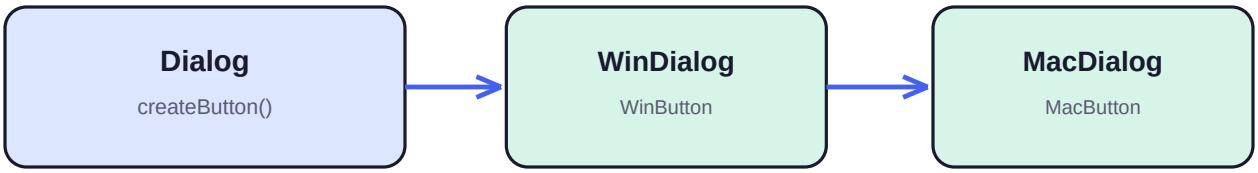
```
class Config:
    _inst = None
    def __new__(cls):
        if cls._inst is None: cls._inst = super().__new__(cls)
        return cls._inst     # every call returns THE instance
```

- Use for: config, connection pools, loggers — truly singular resources.
- Costs: global state in disguise — hidden coupling, painful tests (can't isolate).
- Threads: double-checked locking or module-level init — naive singletons race.

STEP 2/6

Factory Method: Let Subclasses Decide

Define the interface for creation; subclasses pick the class.



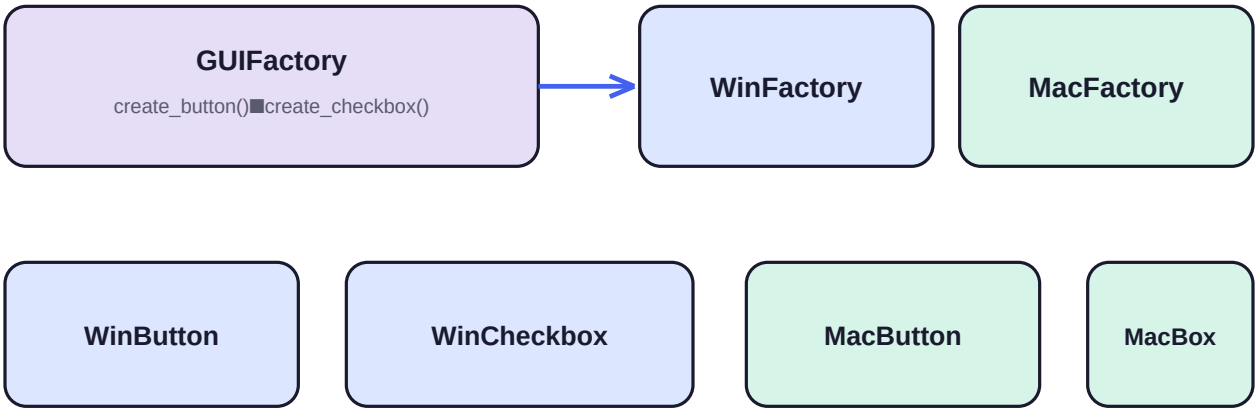

```
class Dialog:
    def render(self): btn = self.create_button()  # subclass decides
class WinDialog(Dialog):
    def create_button(self): return WinButton()
```

- Client code never names a concrete class — the factory method hides it.
- OCP in action: new platform = new Dialog subclass, zero client edits.
- Simple factory (a plain function) covers 80% of cases — reach for the pattern when subclasses vary.

STEP 3/6

Abstract Factory: Families of Products

Create CONSISTENT sets — WinButton with WinCheckbox, never mixed.



WinButton + MacCheckbox together = broken UI. The factory guarantees matching sets.

- Factory method makes ONE product; abstract factory makes a FAMILY.
- Swap entire product lines by swapping one factory object.
- Cost: adding a new product means touching every factory — plan the family upfront.

STEP 4/6

Builder: Step-by-Step Construction

Telescoping constructors are unreadable — build fluently instead.

```
# BEFORE: what does False mean here?
Pizza(size=12, cheese=True, pepperoni=False, gluten_free=True, ...)

# AFTER: readable, optional steps, validated at build()
pizza = (PizzaBuilder().size(12).cheese().gluten_free().build())
```

- Each method returns self (fluent) — steps read like a sentence.
- build() validates the whole: required fields present, combos legal.
- Director: a preset recipe (margherita()) over the same builder steps.
- Immutable result: builder is mutable, the built object is frozen.

STEP 5/6

Prototype: Clone, Don't Rebuild

Copy a configured exemplar instead of constructing from scratch.

```
import copy
base_enemy = Enemy(hp=100, ai='aggressive', sprite=load('orc.png'))
orc2 = copy.deepcopy(base_enemy)   # configured copy, cheap
orc2.hp = 150                      # tweak the clone
```

- Wins when construction is expensive (parsed assets, DB rows, complex graphs).
- Shallow vs deep copy: shared references bite — deepcopy nests, shallow shares.
- Registry of prototypes: ask for 'orc' by name, get a fresh clone — a factory of clones.
- Spawn systems in games: one archetype per enemy type, cloned per spawn.

STEP 6/6

Creational Chooser

Which creation pattern for which pain?

Singleton

Exactly one instance — config, pools — beware global state

Factory Method

Subclass picks the class — platform-specific objects

Abstract Factory

Consistent product families — themed UI kits

Builder

Many optional parameters — fluent, validated construction

Prototype

Expensive to construct — clone a configured exemplar

Start simple: a function returning objects IS a factory. Promote to a pattern only when creation logic itself needs organizing.

STEP ✓I✓

Key Takeaways

Creational Patterns — Remember

- Singleton: one instance, global access — truly singular resources only; it's global state.
- Factory Method: subclasses decide which class to instantiate; clients stay decoupled.
- Abstract Factory: builds consistent FAMILIES of products (never mix WinButton with MacCheckbox).
- Builder: fluent step-by-step construction with validation at build(); kills telescoping constructors.
- Prototype: clone configured exemplars when construction is expensive — mind shallow vs deep copy.

Keep exploring

Pair this guide with the matching video walkthroughs and hands-on labs at interstitiumlabs.dev